

Introduzione

Che co'è l'HPC e a cosa serve

L'HPC (High Performance Computing) è l'utilizzo di supercomputer e tecniche di calcolo parallelo per risolvere problemi computazionali complessi in tempi ragionevoli. A cosa serve così tanta potenza? La risposta è nei problemi detti "Grand Challenge". Si tratta di sfide di notevole impatto scientifico, industriale e sociale che, per essere risolte, richiedono l'elaborazione di moli enormi di dati e simulazioni fisiche estremamente pesanti. Alcuni esempi sono:

- **Aerodinamica e fluidodinamica:** ottimizzazione dei flussi, come fa la Dallara nel settore automobilistico.
- **Energia e Climatologia:** simulazioni di giacimenti per ENI o previsioni meteorologiche su scala globale per l'ECMWF.
- **Intelligenza Artificiale e Big Data:** addestramento di reti neurali complesse e Data Science, supportate da centri come il CINECA in Europa.
- **Sicurezza informatica:** problemi matematici complessi come la fattorizzazione per la crittografia RSA.

L'ecosistema HPC: misurare la potenza e le risorse disponibili

Per supportare queste sfide, l'ecosistema HPC si avvale di diverse tipologie di risorse hardware:

- **Supercomputer on premise:** enormi centri di calcolo fisici. L'elenco dei 500 supercomputer più potenti al mondo è tenuto aggiornato dal progetto **TOP500**.
- **Cloud Computing:** piattaforme commerciali come AWS, Microsoft Azure e Google Cloud che affittano potenza di calcolo.
- **Volunteer Computing:** reti di computer distribuiti in cui i volontari donano la potenza dei propri PC domestici. Un esempio storico è *Folding@home*, che nel 2021 ha raggiunto la mostruosa potenza di 2.43 ExaFlops/s.

Come si misura la potenza

L'unità di misura fondamentale nell'HPC è il **Flops** (Floating Point Operations per Second), ovvero il numero di operazioni in virgola mobile che il sistema può eseguire in un secondo.

Un normale personal computer si aggira intorno ai 100 GFlop/s (10^{11} Flops). I supercomputer in cima alla TOP500 odierna, come il sistema *El Capitan*, superano gli ExaFlops (oltre 10^{18} Flops, ovvero un miliardo di miliardi di operazioni al secondo).

La TOP500 riporta due valori chiave: **Rpeak** (le prestazioni massime teoriche della macchina) e **Rmax** (le prestazioni reali massime ottenute eseguendo un benchmark standard chiamato LINPACK).

Il limite energetico: sostenibilità e raffreddamento

Avere tanta potenza ha un costo energetico esorbitante. Mentre un PC consuma circa 100 Watt, i supercomputer al vertice della classifica assorbono intorno ai 20 MegaWatt (20 milioni di Watt). Poiché il consumo energetico cresce in modo esponenziale insieme alla potenza di calcolo, l'efficienza è diventata una priorità assoluta.

Dal 2016 è stata introdotta la lista **Green500**, che riordina i supercomputer della TOP500 valutandone l'efficienza energetica, misurata in GFlops/Watt.

Per valutare l'efficienza di un intero Data Center si utilizza la metrica **PUE (Power Usage Effectiveness)**, calcolata

come il rapporto tra l'energia totale assorbita dall'edificio e l'energia effettivamente utilizzata dagli apparati informatici (IT).

Tutta l'energia "extra" che non va ai computer viene spesa quasi interamente per il **raffrescamento** delle sale macchine. Un PUE inferiore a 1,8 è considerato un buon livello di efficienza. Un caso di studio eccellente è il data center di Yahoo! vicino alle Cascate del Niagara: sfruttando il clima naturalmente favorevole, ha raggiunto un PUE di 1,08, abbattendo quasi totalmente i costi di condizionamento.

Evoluzione dell'Hardware: la fine della legge di Moore

Per decenni, l'informatica è stata guidata dalla **Legge di Moore** (1965), secondo cui le prestazioni dei microprocessori raddoppiano circa ogni 18 mesi. Tuttavia, intorno al 2005, questa crescita "gratuita" si è scontrata con un muro fisico: rimpicciolire ulteriormente i transistor all'interno della CPU generava un calore impossibile da dissipare. Di conseguenza, la frequenza di lavoro dei processori si è stabilizzata (o "congelata") tra i 2 e i 3 GHz.

Come fa allora la potenza globale a continuare a crescere? L'industria ha cambiato strategia introducendo l'**architettura multicore**: invece di avere un singolo processore potentissimo e velocissimo, si inseriscono più core indipendenti all'interno dello stesso chip.

Questo è un punto di svolta concettuale: per sfruttare le prestazioni dei computer moderni, i vecchi programmi scritti in modo "sequenziale" (un'istruzione dopo l'altra) non bastano più. **Occorre distribuire in parallelo il carico computazionale.**

Livelli di parallelismo e architetture ibride

Eseguire operazioni in parallelo non significa solo usare più core. Esistono diversi "strati" a cui il parallelismo viene applicato nell'hardware moderno:

- **All'interno del Core:** Pipeline, architetture superscalari (più istruzioni per ciclo di clock) ed elaborazione Vettoriale (una singola istruzione agisce su più dati contemporaneamente).
- **All'interno del Nodo:** Più core raggruppati in una CPU (socket) che condividono la cache.
- **Acceleratori:** Dispositivi specializzati (come le GPU) collegati al nodo principale.
- **Tra più Nodi:** I computer formano un "Cluster" comunicando tra loro tramite una rete ad altissima velocità.

Oggi, i computer di fascia altissima sommano tutte queste architetture. Un esempio italiano di eccellenza è il supercomputer **Leonardo**, ospitato al CINECA e posizionato al 10° posto nella TOP500 mondiale. Leonardo unisce nodi "data-centric" (basati su potenti CPU Intel) a migliaia di nodi "booster", ognuno equipaggiato con una CPU e ben 4 potentissime GPU NVIDIA A100. Un'architettura ibrida simile (con nodi CPU, nodi GPU, storage centralizzato e reti ad alta velocità come Infiniband) è presente anche nel cluster di ricerca dell'Università di Parma.

Metriche del Software Parallelo: Speedup e Scalabilità

Prima di scrivere il codice, dobbiamo definire come valutare se il nostro programma parallelo sta funzionando bene. Quando progettiamo il software, scomponiamo il calcolo in diversi *Task* distribuiti sui core. L'obiettivo è minimizzare il tempo di esecuzione.

Le metriche principali sono:

- **Speedup:** È l'accelerazione ottenuta. Si calcola dividendo il tempo impiegato dal programma seriale (T_{seriale}) per il tempo impiegato dalla versione parallela ($T_{\text{parallelo}}$). In un mondo ideale, se uso 4 processori, il programma dovrebbe metterci un quarto del tempo (Speedup ideale = numero di processori).
- **Scalabilità:** È la capacità del software di mantenere una crescita proporzionata dello Speedup all'aumentare delle unità di calcolo.

- **Overhead:** Nel mondo reale, la scalabilità ideale non esiste. Dividere il lavoro introduce dei ritardi (overhead), come il tempo perso per inviare i dati attraverso la rete (comunicazione) o i momenti in cui un core deve aspettare che un altro finisca il suo compito (sincronismo).

I Paradigmi della Programmazione Parallela

Per sfruttare le architetture viste finora, l'HPC utilizza ambienti di sviluppo specifici, compilatori specializzati (come C/C++ e Fortran) e strumenti di analisi delle performance chiamati "profiler". A seconda di come è fatta la memoria dell'hardware, si usano modelli di programmazione diversi:

Memoria Condivisa (Multithreading)

In un sistema SMP (Symmetric Multi Processor), tutti i processori lavorano in modo indipendente ma "vedono" e condividono la stessa memoria RAM.

- **Come funziona:** i task vengono assegnati a entità leggere chiamate *thread*.
- **La comunicazione:** avviene implicitamente. Se il thread A calcola un valore e lo salva in una variabile, il thread B può semplicemente leggere quella stessa variabile. Attenzione: questo richiede meccanismi di "sincronizzazione" rigorosi per evitare che i thread si intralcino modificando lo stesso dato nello stesso momento.
- **Strumenti:** si utilizzano librerie come POSIX Threads o, molto più comunemente, **OpenMP** (Open Multiprocessing), un framework standard supportato nativamente dai compilatori C/C++ e Fortran.

Memoria Distribuita (Message Passing)

In un cluster, ogni computer (o nodo) è una macchina indipendente con la propria RAM privata. Un processore sul nodo 1 non può in alcun modo leggere direttamente la RAM del nodo 2.

- **La comunicazione:** avviene tramite uno scambio esplicito di messaggi attraverso la rete, usando operazioni di invio (`send()`) e ricezione (`recv()`). Questi messaggi possono essere scambiati in modo punto-punto (tra due processi) o globale (tra tutti i processi contemporaneamente).
- **Strumenti:** lo standard assoluto per questo modello è **MPI (Message Passing Interface)**. Esistono implementazioni open source come Open MPI, e soluzioni commerciali come Intel MPI.

Programmazione GPU (Acceleratori)

Le GPU sono nate per renderizzare le immagini a schermo, ma essendo composte da migliaia di piccoli core progettati per processare parallelamente i pixel, si sono rivelate perfette per il calcolo matematico ad alte prestazioni. Dal 2005 si è iniziato a parlare di GP-GPU (General Purpose GPU), rimuovendo persino l'uscita video dalle schede dedicate al calcolo.

- **Come funziona:** la logica del programma (eseguita dalla CPU o host) prende i dati, li copia nella memoria della GPU (device), e le invia una porzione di codice chiamata **Kernel**. La GPU esegue il kernel in modo massicciamente parallelo su tutti i suoi core, per poi restituire i risultati alla CPU.
- **Strumenti:** il modello predominante nel mercato HPC è **CUDA**, creato da NVIDIA (che fornisce il compilatore proprietario `nvcc`). Tuttavia, il mercato sta diversificandosi con le schede AMD (che utilizzano lo stack ROCm) e la linea Intel Arc.